Documentación técnica – Chrono Bomb

Trabajo Práctico Final · Paradigma Orientado a Objetos (3.4.208) · Comisión Lunes tarde · 2° Cuatrimestre 2026 · Docente: Pablo Hernández

Integrantes: Franz Acevedo - Lucas Fernandez - Matias Yujra - Nicolas Caceres · Grupo: Grupo Java1 · Fecha de entrega y defensa: 15/11/2026

1. Descripción general del sistema

1.1 Qué es Chrono Bomb

Chrono Bomb es un juego de escritorio para 2 a 4 jugadores en el mismo teclado, inspirado en Bomberman. Cada jugador controla un personaje en una grilla de 17 × 15 casilleros con paredes indestructibles, bloques rompibles e ítems escondidos. El objetivo es agarrar la corona que está en el centro del mapa y llevarla durante 20 segundos seguidos. Si el portador muere, la corona vuelve al centro y el jugador reaparece a los 3 segundos.

1.2 Reglas principales

- Cada jugador planta bombas que explotan en cruz, se frenan en las paredes y rompen el primer bloque que encuentran.
- Al romper un bloque puede aparecer un ítem (hay 60 repartidos al azar): velocidad (V), golpe aturdidor (G), escudo (E), rango (R), cantidad de bombas (B) y tiempo de detonación (T).
- Una explosión puede detonar otras bombas (reacción en cadena).
- Al morir, el jugador suelta todas sus mejoras en un ítem especial ("?") que cualquiera puede recoger.
- Mientras lleva la corona, el portador va más lento (1,2 casilleros/s), solo puede tener 1 bomba y las mejoras no se aplican (aunque no se pierden).
- Antes de iniciar la partida se elige un modo de juego (nivel), definido en un archivo de texto: Lluvia de Explosiones (4 bombas desde el inicio), Velocidad de la Luz (+4 de velocidad) y Escudo Anti Bombas (todos con escudo).

1.3 Controles

Jugador	Movimiento	Bomba
1	W A S D	Espacio

2	Flechas	Enter
3	I J K L	U
4	Numpad 8 4 5 6	Numpad 0

La tecla ESC durante la partida vuelve al menú principal.

1.4 Contenido del proyecto

El proyecto tiene 18 clases Java en el paquete por defecto y 3 archivos de configuración de niveles:

- Lógica del juego: `GameLoop`, `Jugador`, `Mejoras`, `Bomba`, `Explosion`, `Corona`, `Mapa`, `Item`, `ItemEspecial`, `Movimiento`.
- Configuración: `Config` (constantes) y `Nivel` (lee los modos desde archivo).
- Interfaz gráfica y entrada: `Main`, `MenuPrincipal`, `MenuPartida`, `VentanaJuego`, `PanelJuego`, `ControlTeclas`.
- Archivos de datos: `niveles/lluvia_explosiones.txt`, `niveles/velocidad_luz.txt`, `niveles/escudo_anti_bombas.txt`.

2. Arquitectura y funcionamiento general

2.1 Separación de responsabilidades

El diseño separa tres capas:

- Lógica (modelo): clases que representan las reglas y el estado del juego. No dependen de Swing, salvo `java.awt.Color` y `java.awt.Point` usados como tipos de datos.
- Presentación: `PanelJuego` solo dibuja el estado que le da el `GameLoop`; `VentanaJuego`, `MenuPrincipal` y `MenuPartida` son las ventanas.
- Entrada: `ControlTeclas` traduce el teclado a acciones por jugador y las deja disponibles para el `GameLoop`.

2.2 Flujo de pantallas

1. `Main.main` llama a `SwingUtilities.invokeLater` para crear `MenuPrincipal` en el hilo de Swing.

2. `MenuPrincipal` ofrece "Jugar" (abre `MenuPartida`) y "Salir".
3. `MenuPartida` carga los tres niveles con `Nivel.cargar`, deja elegir cantidad de jugadores (2 a 4) y modo, y muestra los controles. Al iniciar crea una `VentanaJuego`.
4. `VentanaJuego` arma el `PanelJuego` (que a su vez crea el `GameLoop`), registra el `ControlTeclas` como `KeyListener` y arranca el hilo del juego. Con ESC detiene el `GameLoop` y vuelve al menú.

2.3 Hilos del programa

El juego corre con tres tipos de hilos:

- Hilo de Swing (EDT): dibuja la pantalla (`paintComponent`) y recibe los eventos de teclado.
- Hilo `GameLoop`: hace avanzar la lógica unas 60 veces por segundo.
- Un hilo por cada `Bomba`: espera su tiempo de detonación y marca la bomba como explotada.

Como estos hilos comparten datos se usan `volatile`, `synchronized` y `CopyOnWriteArrayList` (ver sección 8.3).

2.4 Un frame del juego

Cada vuelta de `GameLoop.run` calcula `dt` (segundos desde el frame anterior, con tope de 0,05 s), llama a `actualizar(dt)`, pide redibujar y duerme `1000 / FPS` milisegundos. `actualizar(dt)` hace siempre lo mismo y en este orden:

1. Para cada jugador: actualiza temporizadores, lo reaparece si corresponde, procesa la entrada (plantar bomba y mover) y recoge ítems o la corona.
2. Aplica los golpes aturdidores.
3. Actualiza las bombas (explota las que ya terminaron su espera, con reacción en cadena).
4. Actualiza las explosiones y resuelve las muertes.
5. Revisa la condición de victoria.

3. Unidad 1 – Introducción a la POO y UML

3.1 Clases, objetos, atributos y métodos

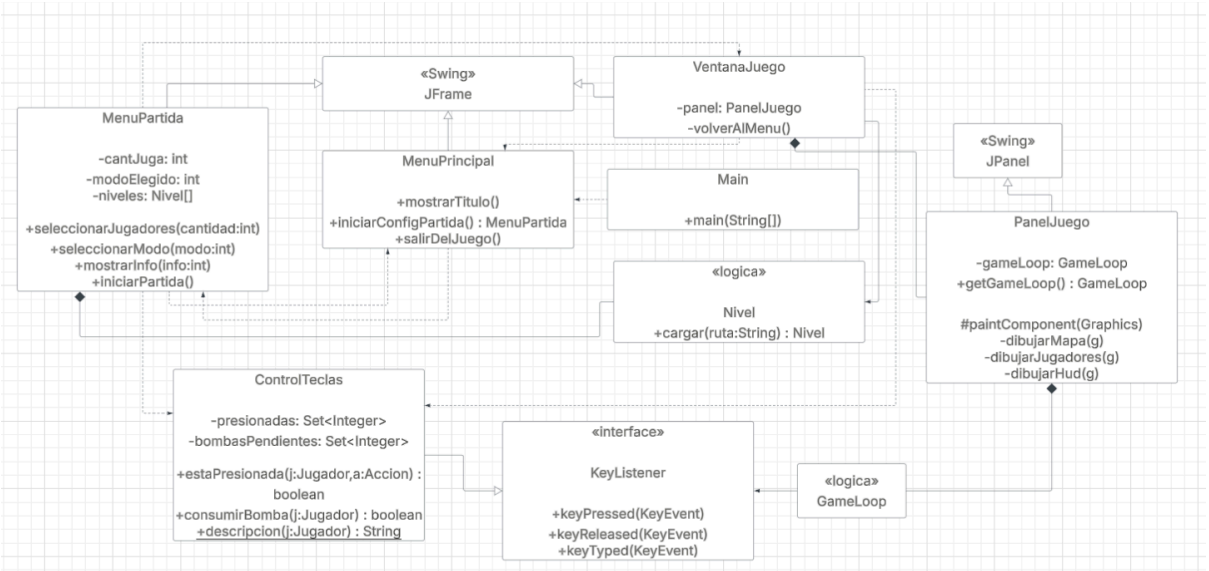
Cada concepto del juego es una clase con sus atributos y métodos. Los objetos se crean durante la partida: el `GameLoop` crea un `Mapa`, una `Corona`, entre 2 y 4 `Jugador`es y un `Movimiento`; los jugadores crean `Bomba`s al plantarlas; al explotar una bomba se crea una `Explosion`; el `Mapa` crea los `Item` al repartir los ítems.

Clase	Responsabilidad
`GameLoop`	Hilo principal de la lógica. Orquesta el frame, las bombas, explosiones, muertes y la victoria.
`Jugador`	Estado y reglas de un jugador: posición decimal, estadísticas efectivas, temporizadores, plantar bomba, recibir explosión, morir y reaparecer.
`Mejoras`	Conjunto de mejoras de un jugador (o contenido de un ítem especial). Permite aplicar, sumar, copiar y reiniciar.
`Item`	Power-up del mapa, con tipo, posición y visibilidad. Elige su tipo al azar según probabilidades.
`ItemEspecial`	Ítem que suelta un jugador al morir; contiene todas sus mejoras.
`Bomba`	Una bomba con su propio hilo que espera el tiempo de detonación.
`Explosion`	Celdas en llamas durante 0,4 s; mata a quien las toque.
`Corona`	Está en el centro hasta que alguien la agarra; recuerda al portador.
`Mapa`	Grilla de paredes, bloques e ítems escondidos.
`Movimiento`	Mueve jugadores sin atravesar paredes, bloques ni bombas.

`Nivel`	Configuración de un modo de juego, leída desde un archivo de texto.
`Config`	Todas las constantes del juego.
`ControlTeclas`	Traduce el teclado a acciones de cada jugador.
`PanelJuego`	Dibuja el estado del juego (mapa, ítems, bombas, explosiones, jugadores y HUD).
`VentanaJuego`, `MenuPrincipal`, `MenuPartida`	Ventanas del programa.
`Main`	Punto de entrada.

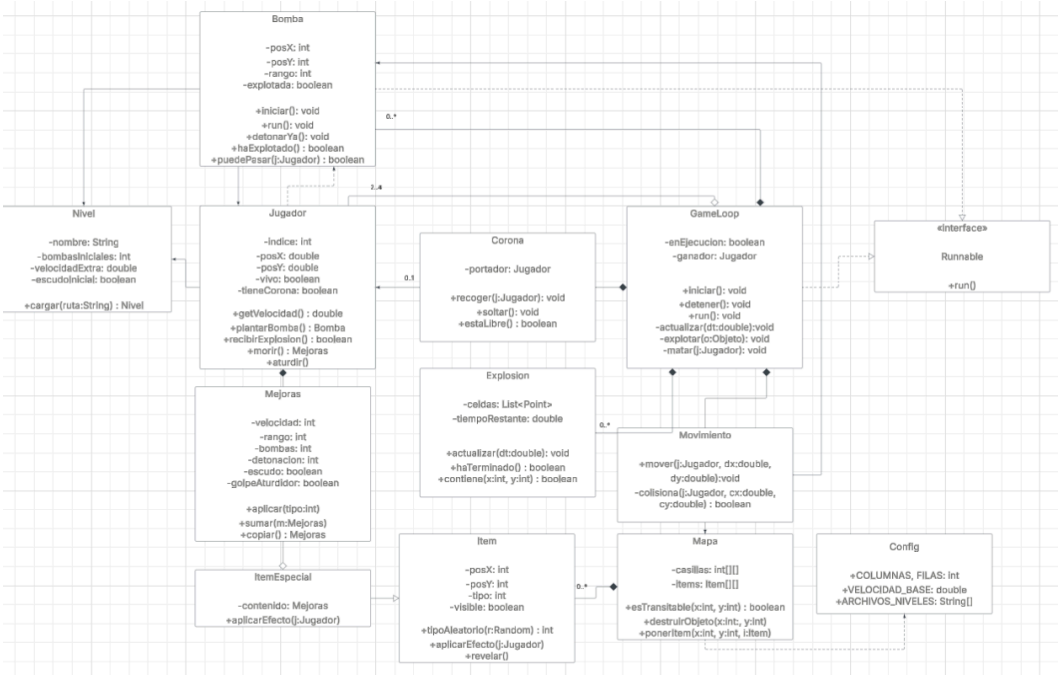
3.2 Diagrama de clases

Comunicación entre clases:



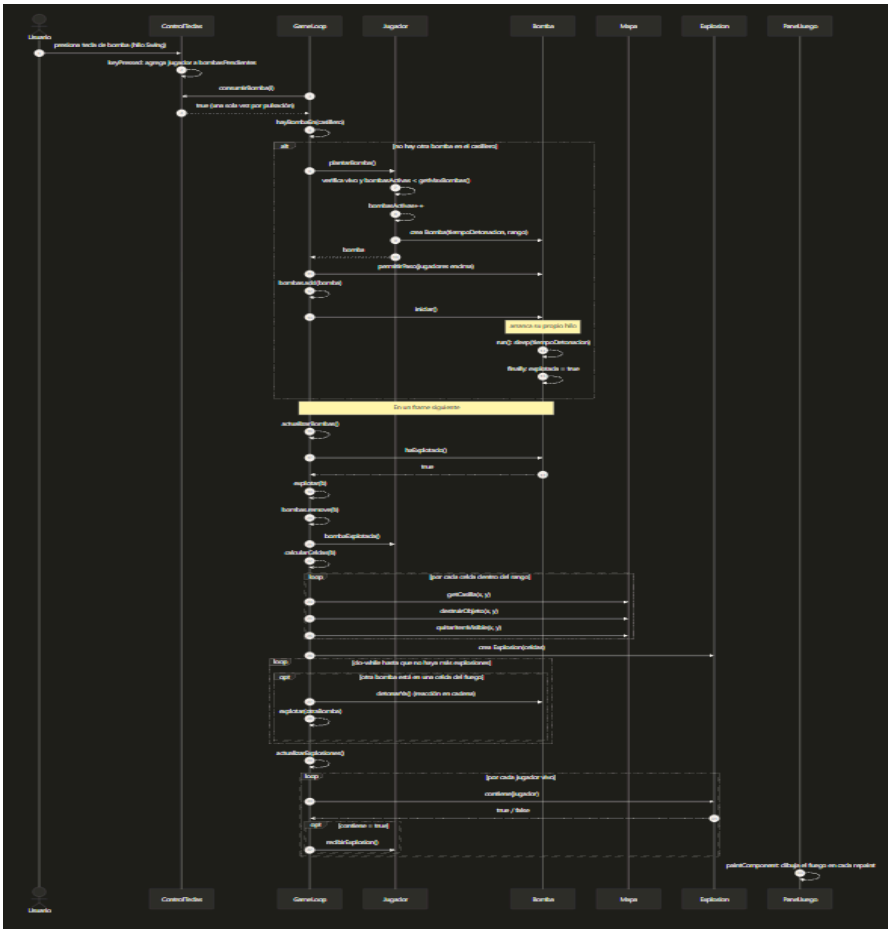
f

Funcionamiento interno del juego (Simulación de partida)::

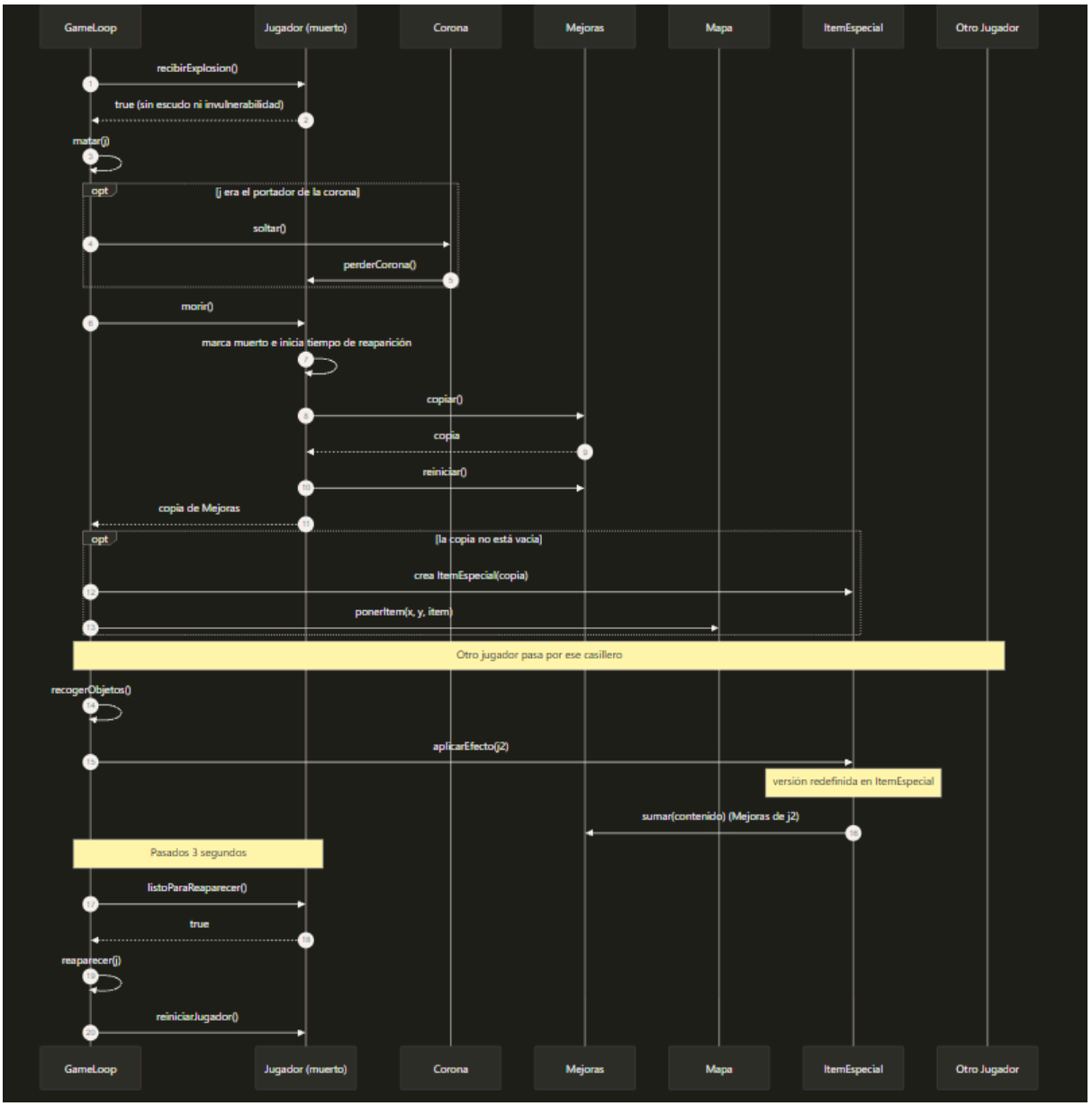


3.3 Diagramas de secuencia

Caso 1: Plantar Bombas



Caso 2: Muertes y mejoras



3.4 Constructores y destructores

Cada clase define su constructor para dejar el objeto en un estado válido:

- `Jugador(indice, color, casilleroX, casilleroY, nivel)`: guarda el spawn (centro del casillero, por eso suma 0,5), copia la posición inicial y aplica el escudo inicial si el nivel lo pide.
- `Bomba(posX, posY, duenio, tiempoDeto, rango)`: guarda los datos y configura el hilo como daemon.
- `GameLoop(cantJugadores, controles, nivel, alRedibujar)`: crea los jugadores en las cuatro esquinas, el `Movimiento` y el resto del estado.
- `Mapa(Random)`: genera la estructura (bordes, pilares y bloques) y reparte los ítems.

- `Item(posX, posY, tipo)`, `ItemEspecial(posX, posY, contenido)`, `Explosion(celdas)`, `Movimiento(mapa, bombas)`, `PanelJuego`, `VentanaJuego` y los menús inicializan sus atributos y componentes.

Java no tiene destructores explícitos: la memoria de los objetos que dejan de usarse la libera el recolector de basura (Garbage Collector). Lo equivalente que se hizo en este proyecto es liberar los recursos que el recolector no maneja:

- `Nivel.cargar` cierra el archivo en un bloque `finally`.
- `GameLoop.detener()` y el `finally` de `run()` cancelan los hilos de todas las bombas (`cancelarBombas`).
- `VentanaJuego.volverAlMenu()` y `MenuPartida.iniciarPartida()` llaman a `dispose()` para liberar los recursos gráficos de la ventana.
- Los hilos de las bombas y del `GameLoop` son daemon, así que no mantienen vivo el programa al cerrarlo.

3.5 Tipos primitivos y wrapper

- Primitivos: `int` (posiciones de casilleros, rangos, cantidades), `double` (posición decimal de los jugadores, velocidades y tiempos), `boolean` (`vivo`, `tieneCorona`, `explotada`, `escudo`), `long` (`System.nanoTime()` en `Bomba.inicio`).
- Wrappers: `Set<Integer>` en `ControlTeclas` (`presionadas`, `bombasPendientes`). Las colecciones genéricas no aceptan primitivos, así que `presionadas.add(codigo)` hace autoboxing de `int` a `Integer` y `presionadas.contains(...)` / `remove(...)` lo repiten. Además se usan los métodos de conversión `Integer.parseInt`, `Double.parseDouble` y `Boolean.parseBoolean` en `Nivel.interpretarLinea`.

4. Unidad 2 – Funcionamiento del lenguaje orientado a objetos

4.1 Estructura básica de un programa Java y buenas prácticas

El programa arranca en `Main.main(String[] args)`. Todo está en el paquete por defecto, una clase por archivo y con el mismo nombre del archivo. Las buenas prácticas aplicadas:

- Nombres descriptivos en español y con convención Java: clases en `UpperCamelCase`, métodos y atributos en `lowerCamelCase`, constantes en `MAYUSCULAS_CON_GUION_BAJO`.
- Constantes agrupadas en `Config` (clase `final` con constructor privado), sin números mágicos repartidos por el código.
- Atributos privados con getters; los atributos que no cambian son `final`.

- Métodos cortos con una sola tarea (por ejemplo `actualizar` delega en cinco métodos, uno por etapa).
- Comentarios Javadoc en las clases y en los métodos no triviales.
- Uso de `@Override` al redefinir métodos.

4.2 Tipos primitivos y de referencia, y casteo

Se usan tipos primitivos para el estado numérico y tipos de referencia para los objetos (`Jugador`, `Bomba`, `Mapa`, `String`, `Color`, `Point`, arreglos y colecciones). Casteos presentes en el código:

- `(int) Math.floor(posX)` en `Jugador.getCasilleroX/Y`: convierte la posición decimal al casillero entero.
- `(long) (tiempoDeto * 1000)` en `Bomba.run`: convierte segundos a milisegundos para `Thread.sleep`.
- `(Graphics2D) g0` en `PanelJuego.paintComponent`: casteo de referencia (downcasting) para usar las funciones de dibujo avanzadas.
- `(int) Math.round(...)` y `(int) (j.getTiempoInvulnerable() * 10)` en `PanelJuego.dibujarJugadores`: posición en pantalla y efecto de parpadeo.

4.3 Arreglos

- `int[][] casillas` y `Item[][] items` en `Mapa`: matrices de primitivos y de referencia que representan la grilla.
- `Jugador[] jugadores` en `GameLoop`: arreglo de referencias, de tamaño 2 a 4.
- `Nivel[] niveles` en `MenuPartida`: un nivel por cada archivo.
- `String[] ARCHIVOS_NIVELES` en `Config`: rutas de los archivos de nivel.
- `int[][] TECLAS` en `ControlTeclas`: fila = jugador, columna = acción (arriba, abajo, izquierda, derecha, bomba).
- Arreglos paralelos en `Item` (`PROBABILIDAD`, `LETRAS`, `COLORES`), indexados por el tipo de ítem.
- `int[][] esquinas` y `int[][] direcciones` en `GameLoop`: posiciones iniciales de los jugadores y las cuatro direcciones de la explosión.

5. Unidad 3 – Clases y objetos

5.1 Herencia

- `ItemEspecial extends Item`: es un ítem que, en lugar de un único efecto, contiene todas las mejoras de un jugador muerto. Su constructor llama a `super(posX, posY, ESPECIAL)` y luego a `revelar()` porque nace visible.
- `MenuPrincipal`, `MenuPartida` y `VentanaJuego` extienden `JFrame` y llaman a `super(titulo)`.
- `PanelJuego extends JPanel`.

5.2 Sobreescritura y sobrecarga

Sobreescritura (presente):

- `ItemEspecial.aplicarEfecto(Jugador)` redefine `Item.aplicarEfecto(Jugador)`.
- `Bomba.run()` y `GameLoop.run()` implementan `Runnable.run()`.
- `PanelJuego.paintComponent(Graphics)` redefine el método de `JPanel`.
- `ControlTeclas.keyPressed`, `keyReleased` y `keyTyped` implementan `KeyListener`; en `VentanaJuego` una clase anónima redefine `KeyAdapter.keyPressed`.

Sobrecarga de métodos y de constructores: ver la sección 10 (punto a reforzar).

5.3 Modificadores de acceso

- `private`: casi todos los atributos y los métodos auxiliares (por ejemplo `GameLoop.explotar`, `Movimiento.colisiona`, `Nivel.interpretarLinea`). Es el encapsulamiento: el estado solo se modifica a través de métodos.
- `public`: la interfaz de cada clase (constructores, getters y operaciones como `Jugador.plantarBomba`).
- `protected`: `PanelJuego.paintComponent`, heredado de `JPanel`.
- `default` (paquete): no se usa explícitamente; ver sección 10.

5.4 Clase abstracta e interfaz

- Interfaces implementadas, con su contrato: `Runnable` (`run()`, contrato para ejecutar código en un hilo; la implementan `GameLoop` y `Bomba`) y `KeyListener` (`keyPressed`, `keyReleased`, `keyTyped`; la implementa `ControlTeclas`).
- Clase abstracta propia e interfaz propia: ver sección 10 (punto a reforzar).

5.5 Polimorfismo

En `GameLoop.recogerObjetos` se llama a `item.aplicarEfecto(j)` sobre una referencia de tipo `Item`. Si el objeto es un `Item` común aplica una única mejora (`Mejoras.aplicar(tipo)`); si es un `ItemEspecial` suma todo su contenido (`Mejoras.sumar(contenido)`). El `GameLoop` no pregunta de qué tipo es: la decisión la toma el objeto. Lo mismo ocurre con `Runnable`: la clase `Thread` ejecuta `run()` sin saber si es un `GameLoop` o una `Bomba`.

5.6 Uso de super() y this()

- `super(...)`: en `ItemEspecial` (llama al constructor de `Item`) y en los tres `JFrame`.

- `this(...)`: ver sección 10 (punto a reforzar). Dentro del código se usa `this.` para desambiguar atributos y parámetros, y `this::repaint` en `PanelJuego` como referencia a método.

5.7 Modificador static

- Constantes de clase: todo `Config`, `Item.VELOCIDAD`...`ESPECIAL`, `Mapa.VACIO/PARED/BLOQUE` y `ControlTeclas.ARRIBA`...`BOMBA`.
- Métodos estáticos: `Main.main`, `Nivel.cargar(ruta)` (fábrica que devuelve un `Nivel` ya armado), `Item.tipoAleatorio(Random)` y `ControlTeclas.descripcion(jugador)`.
- Datos compartidos por todas las instancias: `Item.PROBABILIDAD`, `LETRAS`, `COLORES` y `ControlTeclas.TECLAS`.

5.8 Patrones GRASP y principios SOLID identificados

GRASP:

- Experto en información: cada clase calcula lo que sabe. `Jugador` calcula sus estadísticas efectivas (`getVelocidad`, `getRango`, `getMaxBombas`, `getTiempoDetonacion`) porque tiene las `Mejoras`, el `Nivel` y si lleva la corona. `Mapa` sabe qué casilleros son transitables.
- Creador: `Jugador.plantarBomba()` crea la `Bomba` porque tiene los datos para inicializarla (posición, rango, tiempo y dueño). `Mapa` crea los `Item` que reparte.
- Controlador: `GameLoop` recibe los eventos del sistema (entrada y paso del tiempo) y coordina a los demás objetos sin contener las reglas internas de cada uno.
- Bajo acoplamiento y alta cohesión: `PanelJuego` solo dibuja y recibe el `GameLoop` y un `Runnable` para redibujar; el `GameLoop` no conoce a `PanelJuego`.
- Fabricación pura: `Movimiento` no representa un objeto del dominio; existe para concentrar el manejo de colisiones.

SOLID:

- Responsabilidad única (SRP): `Movimiento` solo mueve, `PanelJuego` solo dibuja, `Config` solo guarda constantes, `Mejoras` solo agrupa mejoras, `Nivel` solo lee y guarda un modo de juego. El propio código lo menciona al separar `Movimiento` del `GameLoop`.
- Abierto/cerrado (OCP): se pueden agregar modos de juego nuevos creando un `.txt` y sumando su ruta en `Config.ARCHIVOS_NIVELES`, sin modificar las clases existentes. Un nuevo tipo de ítem con efecto distinto se agrega por herencia, como `ItemEspecial`.
- Inversión de dependencias : `GameLoop` depende de un `Runnable alRedibujar` y no de la clase `PanelJuego`.

5.9 Elementos opcionales

- Clases inmutables `Config` es `final` y sin instancias; varios atributos son `final` (por ejemplo `Explosion.celdas`, `Jugador.indice`).
- Clases internas: se usa una clase anónima (`KeyAdapter` en `VentanaJuego`) y expresiones lambda para los `ActionListener` de los botones.
- Tipos enumerados: no se usan; ver sección 10.

6. Unidad 4 – Colecciones

6.1 Colecciones genéricas

Colección	Dónde	Para qué
`List<Bomba>` (`CopyOnWriteArrayList`)	`GameLoop.bombas`, `Movimiento`	Bombas activas; se recorre para dibujar mientras otro hilo agrega o quita.
`List<Explosion>` (`CopyOnWriteArrayList`)	`GameLoop.explosiones`	Explosiones activas.
`List<Point>` (`ArrayList`)	`Explosion.celdas`, `Mapa.repartirItems`, `GameLoop.calcularCeldas`	Celdas del fuego y bloques candidatos a esconder un ítem.
`Set<Jugador>` (`HashSet`)	`Bomba.atravesadores`	Jugadores que pueden salir caminando de la bomba que plantaron encima.
`Set<Integer>` (`HashSet`)	`ControlTeclas.presionadas`, `bombasPendientes`	Teclas presionadas y bombas pedidas; el `Set` ignora repetidos.

6.2 Cursores y recorridos

Se usa el `for-each` para recorrer colecciones y arreglos (`for (Bomba b : bombas)`, `for (Jugador j : jugadores)`, `for (Point p : celdas)`). Además, `removeIf` recorre internamente con un iterador: `explosiones.removeIf(Explosion::haTerminado)` y `atravesadores.removeIf(j -> !j.tocaCasillero(...))`.

6.3 Ordenamiento

`Collections.shuffle(bloques, azar)` en `Mapa` mezcla los bloques para elegir al azar dónde esconder los 60 ítems. El ordenamiento con `Comparable` o `Comparator` y el uso de `Map` no están implementados; ver sección 10.

7. Unidad 5 – Manejo de excepciones

7.1 try, catch y finally

Lugar	Qué captura	Para qué
`Nivel.cargar`	`FileNotFoundException`, `IOException`, `NumberFormatException`	Si el archivo falta, no se puede leer o tiene un número inválido, guarda un mensaje en `error` y el nivel usa valores normales. El `finally` cierra el `BufferedReader`, haya o no error.
`GameLoop.run`	`InterruptedException`, `RuntimeException`	La primera es la forma normal de cortar el bucle (`detener()` interrumpe el sueño). La segunda muestra el error en consola en lugar de dejar morir el hilo en silencio. El `finally` apaga el loop, cancela las bombas

		y deja la pantalla final dibujada.
`Bomba.run`	`InterruptedException`	Si la interrumpen (otra explosión o fin de partida) no es un error. El `finally` siempre marca la bomba como explotada.
`MenuPartida.iniciarPartida`	`RuntimeException`	Si no se puede abrir la ventana del juego, muestra un `JOptionPane` y vuelve al menú. El `finally` cierra la pantalla de configuración.
`ControlTeclas.descripcion`	`ArrayIndexOutOfBoundsException`	Si se pide el jugador 5 o más, devuelve "Sin controles asignados".

7.2 throw, throws y excepción propia

El código actual no usa `throw` ni `throws` y no define una excepción propia: solo captura excepciones de la librería. Ver sección 10 (punto a reforzar).

8. Unidad 6 – Entrada/Salida, Swing, archivos e hilos

8.1 Archivos

Cada modo de juego es un archivo de texto en `niveles/` con líneas `clave=valor` (las que empiezan con `#` son comentarios). `Nivel.cargar` lo lee con `FileReader` (con codificación UTF-8) envuelto en un `BufferedReader`, línea por línea, y `interpretarLinea` separa clave y valor con `indexOf('=')` y asigna el campo con un `switch`.

Clave	Tipo	Significado

`nombre`	texto	Nombre del modo que se muestra en el menú.
`descripcion`	texto	Descripción del modo.
`bombasIniciales`	entero	Bombas que se pueden plantar a la vez al empezar (1 por defecto).
`velocidadExtra`	decimal	Casilleros por segundo que se suman a la velocidad base (3).
`escudoInicial`	booleano	Si todos aparecen y reaparecen con escudo.

Las claves desconocidas y las líneas sin `=` se ignoran. Si el archivo falta o tiene un error, `Nivel` conserva sus valores por defecto (partida normal) y `MenuPartida` muestra un aviso.

8.2 Interfaz gráfica y eventos

- Ventanas y componentes Swing: `JFrame`, `JPanel`, `JButton`, `JRadioButton` con `ButtonGroup`, `JLabel`, `JTextArea`, `JOptionPane`, y administradores de diseño `GridLayout`, `FlowLayout` y `BorderLayout`.
- Eventos de acción: los botones y radio buttons registran `ActionListener` con expresiones lambda (por ejemplo, "Iniciar partida" llama a `iniciarPartida()`).
- Eventos de teclado: `PanelJuego` registra el `ControlTeclas` (implementa `KeyListener`) y un `KeyAdapter` anónimo que detecta ESC. `ControlTeclas` ignora la repetición automática del teclado con `presionadas.add(codigo)`, que devuelve false si la tecla ya estaba apretada.
- Dibujo: `PanelJuego.paintComponent` usa `Graphics2D` con suavizado de bordes y dibuja en este orden: mapa, ítems, bombas, explosiones, corona libre, jugadores, HUD y pantalla final si hay ganador.
- La posición de los jugadores es vectorial (decimal, en casilleros) y el movimiento está normalizado para que en diagonal no vaya más rápido.

8.3 Hilos y sincronización

- `GameLoop implements Runnable`: `iniciar()` crea el `Thread` "GameLoop", lo marca daemon y lo arranca. `detener()` apaga `enEjecucion` e interrumpe el sueño entre frames.
- `Bomba implements Runnable`: cada bomba crea su propio `Thread` "Bomba" (daemon). `run()` hace `Thread.sleep` del tiempo de detonación; `detonarYa()` y `cancelar()` lo interrumpen con `interrupt()`.
- `volatile`: `GameLoop.enEjecucion` y `ganador`, y `Bomba.explotada` e `inicio`, que un hilo escribe y otro lee.
- `synchronized`: todos los métodos de `ControlTeclas` que tocan los conjuntos de teclas, porque el hilo de Swing escribe y el `GameLoop` lee.
- `CopyOnWriteArrayList`: `bombas` y `explosiones` se pueden recorrer para dibujar mientras otro hilo agrega o quita elementos, sin `ConcurrentModificationException`.
- `SwingUtilities.invokeLater` en `Main` crea la interfaz en el hilo de Swing.